Laboratorium 06 – Budowanie wyglądu aplikacji w Flutterze

W tym laboratorium nauczysz się korzystać z podstawowych widgetów odpowiedzialnych za **wygląd i układ elementów na ekranie**. Flutter opiera się na widgetach, a budowanie UI to nic innego jak układanie ich w odpowiedniej hierarchii.

---

1. Dodanie nagłówka AppBar ze stylami

Każda aplikacja może mieć pasek nagłówka (AppBar). Możemy go stylizować:

```
Scaffold(
  appBar: AppBar(
    title: const Text(
      'Moja Aplikacja',
      style: TextStyle(
        fontSize: 24,
        fontWeight: FontWeight.bold,
        color: Colors.white,
      ),
    ),
    backgroundColor: Colors.deepPurple,
    centerTitle: true, // wyśrodkowanie tytułu
  ),
  body: const Center(child: Text('Treść aplikacji')),
)
```

👉 **Wyjaśnienie:**

- `title` – co wyświetla się w pasku.
- `style` – zmiana wyglądu tekstu (rozmiar, pogrubienie, kolor).
- `backgroundColor` – kolor tła `AppBar`.
- `centerTitle` – wyśrodkowuje tytuł (przydatne np. na iOS).

---

2. Budowa kontenera

`Container` to prostokąt, który można stylizować (kolor, marginesy, obramowania).

```
```

```
Container(
  width: 200,
  height: 100,
  padding: const EdgeInsets.all(16), // wewnętrzny odstęp
  margin: const EdgeInsets.all(20),  // zewnętrzny odstęp
  decoration: BoxDecoration(
    color: Colors.amber,
    borderRadius: BorderRadius.circular(12),
    border: Border.all(color: Colors.black, width: 2),
  ),
  child: const Text('To jest kontener'),
)
```

👉 **Wyjaśnienie:**

- `width` i `height` określają wymiary.
- `padding` – odstęp wewnątrz kontenera (od brzegu do zawartości).
- `margin` – odstęp na zewnątrz kontenera.
- `decoration` – pozwala nadać kolor, zaokrąglenia, obramowania.

---

3. Widgety do ustawiania elementów

• Center

Umieszcza jeden widget na środku ekranu.

```
Center(
child: Text('Jestem wyśrodkowany'),
)
```

• SizedBox

Dodaje pustą przestrzeń albo ustala rozmiar widgetu.

```
Column(
 children: [
   const Text('Pierwszy'),
   const SizedBox(height: 20), // odstęp 20 px
   const Text('Drugi'),
 ],
)
```

• Column i Row

Układają elementy pionowo (**Column**) lub poziomo (**Row**).

```
Row(
 mainAxisAlignment: MainAxisAlignment.spaceEvenly,
 children: [
   const Text('A'),
   const Text('B'),
   const Text('C'),
 ],
)
```

• Expanded

Rozciąga widget, aby wypełnił dostępne miejsce.

```
Row(
  children: [
    Expanded(
      child: Container(color: Colors.red, height: 50),
    ),
    Expanded(
      flex: 2, // zajmuje 2 razy więcej miejsca
      child: Container(color: Colors.blue, height: 50),
    ),
  ],
)
```

👉 **Wyjaśnienie:**

- `Expanded` działa tylko wewnątrz `Row` lub `Column`.
- `flex` ustala proporcje podziału przestrzeni.

Zadania Praktyczne 💻

1. **AppBar ze stylem**

   Stwórz aplikację z własnym AppBar. W tytule użyj **innej czcionki** i wyśrodkuj napis. Kolor AppBar ustaw według własnego pomysłu.

2. **Kontener jako karta**

   Zbuduj Container, który wygląda jak prostokątna karta z tekstem w środku (kolor tła + zaokrąglone rogi). Dodaj marginesy i padding tak, żeby tekst nie był przy samych krawędziach.
3. **Układ pionowy**

   Użyj Column, aby wyświetlić 3 elementy: napis, obrazek (np. Image.network), a na dole przycisk. Ustaw je tak, żeby wszystko było wyśrodkowane na ekranie.
4. **Układ poziomy z Expanded**

   Stwórz Row z trzema kolorowymi Containerami. Pierwszy niech zajmuje mało miejsca, a drugi i trzeci po równo całą resztę szerokości ekranu.

Przydatne Linki 🔗

- **Layout w Flutterze:** https://docs.flutter.dev/ui/layout
- **AppBar:** https://api.flutter.dev/flutter/material/AppBar-class.html
- **Container:** https://api.flutter.dev/flutter/widgets/Container-class.html
- **Expanded:** https://api.flutter.dev/flutter/widgets/Expanded-class.html